

Proof Buddy — System Architecture

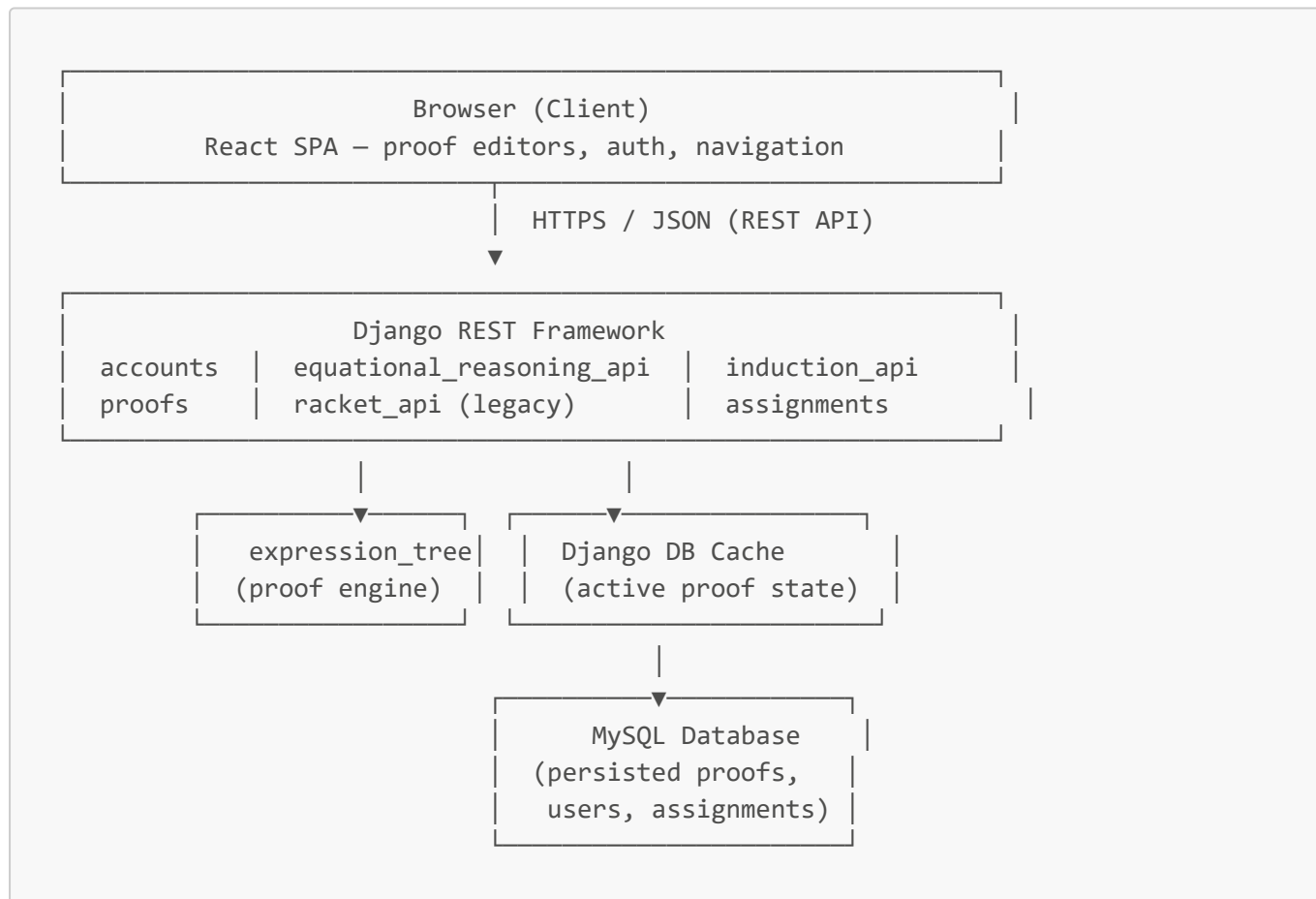
1. Overall Purpose

Proof Buddy is a web-based educational tool that allows students and instructors to write, check, and save mathematical proofs interactively. It was developed for use in courses at Drexel University. The system currently supports three distinct styles of proof:

- **Equational Reasoning:** Prove that two expressions are equal by rewriting each side step-by-step until both sides converge on the same expression.
- **Mathematical Induction:** Prove statements about integers or lists by establishing a base case and a leap (inductive) step.
- **Natural Deduction** (legacy mode): Propositional and first-order logic proof checking. This mode exists in the codebase as older pages ([NaturalDeductionPropositionalLogic.js](#), [NaturalDeductionFirstOrderLogic.js](#)) and a legacy racket engine ([ERRacket.js](#)).

The expressions users work with are written in a Racket-like s-expression syntax (parenthesized prefix notation), and the proof engine checks work symbolically using an internal typed abstract-syntax-tree (AST) representation.

2. Major Subsystems



2.1 React Frontend (client/)

A single-page application (SPA) built with React 18 and React Router v6 (HashRouter). Communicates with the Django backend exclusively through REST API calls made with Axios. Handles rendering proof lines, node selection via arrow-key navigation, definition management, and authentication flows.

2.2 Django Backend (django_server/)

A Python/Django 5 application exposing a REST API through Django REST Framework (DRF). Contains multiple Django "apps", each responsible for a distinct domain:

- **accounts**: User authentication, registration, email verification, token management.
- **equational_reasoning_api**: All endpoints for the equational reasoning proof mode.
- **induction_api**: All endpoints for the mathematical induction proof mode.
- **proofs**: Legacy proof storage, definition and generic management.
- **racket_api**: Legacy proof validation endpoints.
- **assignments**: Instructor course/assignment management and student submission tracking.

2.3 Proof Engine (expression_tree/)

A pure-Python library within the Django codebase. It is the intellectual core of the system. It parses Racket-like expressions, builds typed ASTs, and evaluates whether a claimed rule application is valid. It is completely stateless with respect to the web layer — it receives expression strings and rule strings, and returns results and errors. The active proof object during a session is serialized with `dill` and stored in Django's database-backed cache.

2.4 MySQL Database

Stores all persistent data: user accounts, saved proofs, proof lines, definitions, generics, terms, and assignments. Django ORM models define the schema. Migrations are used to evolve the schema.

2.5 Django Cache Layer

Uses Django's database-backed cache (configured in `settings.py`). During an active proof session, a serialized `TwoSidedProof` or `IndProof` Python object is stored in the cache keyed by the user's authentication token. This avoids rebuilding the full proof tree from the database on every API call. Cache entries expire after 30 minutes of inactivity.

2.6 Nginx + Docker Deployment

In production/Docker mode, Nginx acts as a reverse proxy routing API requests to Gunicorn (Django) and serving the built React app as static files. Docker Compose orchestrates three containers: `django_server`, `nginx`, and `client`.

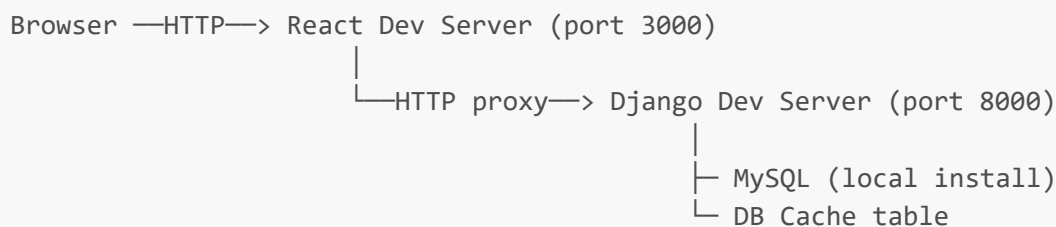
3. Technology Stack

Layer	Technology
Frontend framework	React 18
Frontend routing	React Router v6 (HashRouter)

Layer	Technology
Frontend HTTP client	Axios
Frontend styling	Bootstrap 5 + React-Bootstrap + SCSS
Backend framework	Django 5 + Django REST Framework
Backend expression engine	Custom Python (expression_tree package)
Symbolic math	SymPy
Object serialization	dill
Database	MySQL (via mysqlclient)
Caching	Django DB Cache
Reverse proxy	Nginx
Containerization	Docker + Docker Compose
Authentication	DRF Token Authentication (per-request token in HTTP header)

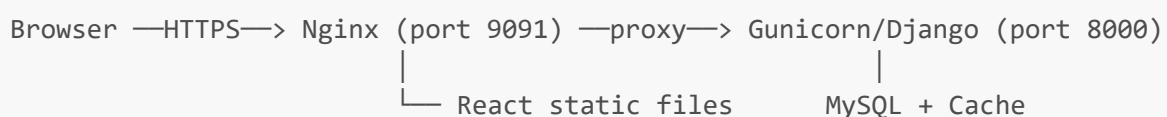
4. how the Frontend, Backend, Database, and Deployment Stack Interact

Development mode



The React dev server proxies API calls to Django. There is no Nginx in local development.

Production (Docker) mode



The built React app is served as static files from Nginx. API calls to `/api/v1/...` are proxied to the Django container.

5. Request Flow Through the System

This example traces a "apply rule" action in the equational reasoning proof editor:

1. **User action:** The student selects a subexpression in a proof line by clicking (or using arrow keys), types a rule (e.g., `eval +`), and clicks "GENERATE & CHECK".
2. **Frontend:** `EquationalReasoningNew.js` calls `equationalService.applyRule(payload)` which sends a `POST` to `/api/v1/equational/apply-rule`.
3. **Django routing:** `django_server/urls.py` routes the request to `equational_reasoning_api/urls.py`, which dispatches to the `apply_rule` view function.
4. **View function** (`equational_reasoning_api/views.py`):
 - Reads the user's auth token to identify them.
 - Retrieves the serialized `TwoSidedProof` from the Django cache.
 - Reloads saved proof lines from the MySQL database into the in-memory proof object.
 - Calls `proof_obj.LHS.addProofLine(currentRacket, rule, startPosition)` (or RHS).
5. **Proof Engine** (`expression_tree/`):
 - `addProofLine()` in `ERProofEngine.py` creates a new `ERProofLine`.
 - The expression string is tokenized and parsed into a Node tree by `Parser.py`.
 - `expressionDefinition.py` assigns types to every node using `labelTree()`.
 - The Decorator further propagates type information.
 - The rule string is looked up in the `ruleSet` dictionary.
 - The rule's `isApplicable()` and `insertSubstitution()` methods are called on the selected node.
 - Any errors are recorded in the proof line's `errLog`.
 - The resulting expression tree is serialized to a JSON dictionary by `makeJson()`.
6. **Back in the view:** The result (new racket string, JSON tree, errors, result node ID) is saved to the MySQL database as an `EquationalProofLine` record. The updated proof object is re-serialized to the cache.
7. **Response:** JSON is returned to the frontend with `{isValid, racket, jsonTree, rule, resultNodeId, errors}`.
8. **Frontend update:** React state is updated, the new proof line is rendered using `PersistentPad`, and the result node is highlighted.

6. How Proofs Are Represented Internally

6.1 Expression Trees (Nodes)

Every mathematical expression is represented as a tree of `Node` objects (defined in `ERCommon.py`). Each node has:

- `data`: The string token at this position (e.g., `"+"`, `"5"`, `"cons"`, `"("`)
- `_type`: A `RacType` object describing the type (e.g., `INT`, `BOOL`, `(INT, INT) > INT`)
- `children`: A list of child `Node` objects (for function applications)
- `parent`: Reference to the parent node
- `startPosition`: Integer index into the original expression string, used as a stable node identifier across frontend/backend communication

- **name**: The computed semantic value (e.g., an integer, a lambda, or an **ERobj** descriptor)

For example, the expression **(+ 2 3)** becomes a tree:

```
Node(data="(", type=INT)
├── Node(data="+", type=(INT,INT)>INT)
├── Node(data="2", type=INT)
└── Node(data="3", type=INT)
```

6.2 Proof Objects

An **ERProof** (single-sided) contains a list of **ERProofLine** objects, each representing one step in the proof. A **TwoSidedProof** contains two **ERProof** objects — one for the left-hand side (LHS) and one for the right-hand side (RHS). An **IndProof** contains a **TwoSidedProof** for the base case and another for the leap step, plus metadata about the induction variable, anchor value, and leap variable.

6.3 JSON Representation Sent to Frontend

The **makeJson()** function converts an expression tree into a flat dictionary usable by the React frontend. Keys are integer start positions (node IDs), values are objects with **data**, **children**, **parent**, **leftSib**, **rightSib**. This structure allows the frontend's **PersistentPad** component to implement keyboard navigation through the tree.

7. How Proof Validation Works

Proof validation is rule-based. At each step, the user claims that some subterm of the current expression can be rewritten according to a named rule.

7.1 Rule Types

Defined in **ERRuleset.py**:

Rule type	Description
BUILT_IN	Native Racket procedures: cons , first , rest , null? , zero? , = , > , etc.
MATH	Symbolic algebraic operations using SymPy: + , - , * , quotient , remainder , expt
AXIOM	First-principles identities: cons-first-rest , null?-null , zero?-0 , etc.
DEFINITION	User-defined recursive functions (UDFs)
LEMMA	Internal lemmas (reusable proven identities)
IH	Inductive Hypothesis (only valid in Induction proofs)

7.2 The Rule Application Process

1. The rule string from the user is parsed to extract the rule name and optional label (e.g., **"apply length"** or **"eval +"** or **"rewrite IH"**).

- For documentation for the use of Lemmas see: [Lemma Documentation](#)
- 2. The selected node in the expression tree is located using `startPosition`.
- 3. The rule object's `isApplicable()` method checks whether the rule can be applied to the selected node.
- 4. If applicable, `insertSubstitution()` computes the replacement subtree.
- 5. The selected node in the expression tree is replaced with the result using `replaceWith()`.
- 6. For `eval` rules with an explicit substitution expression, `isMatch()` first checks structural equality between the provided expression and the selected node.

7.3 Symbolic Math via SymPy

The `Math` rule subclasses in `ERRuleset.py` use SymPy's `simplify()` to handle arithmetic symbolically. This means $(+ (* 2 n) (* 2 k))$ can be simplified to $(* 2 (+ n k))$ even when `n` and `k` are generic variables.

7.4 Completion Checking

`TwoSidedProof.checkComplete()` verifies that:

1. Neither the LHS nor RHS side has a blank last-used line.
2. The last non-blank expression on the LHS is structurally identical (as a string) to the last non-blank expression on the RHS.
3. No lines have their `hide_expression` or `hide_justification` flags set (used by instructors to create partial proofs for students).

For induction, completion requires both the base case and leap step sub-proofs to be individually complete.

8. Authentication and Authorization

The system uses DRF Token Authentication. On login, the backend issues an opaque token that the frontend stores in a cookie via `js-cookie`. Every subsequent API request includes this token in the `Authorization: Token <value>` header. The backend's `@api_view` decorators with `authentication_classes=[TokenAuthentication]` and `permission_classes=[IsAuthenticated]` enforce that only authenticated users can access proof endpoints.

Instructors have an `is_instructor` flag on their `Account` model record. Instructor-only features (assignment creation, visibility toggling on proof lines) are gated by this flag in the backend views.